

1η ΣΑΕΚ ΗΡΑΚΛΕΙΟΥ

ΜΑΘΗΜΑ: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ (C++)

Φύλλο Ανάθεσης Εργασίας Αριθμ. 8Α Προγραμματίζοντας με Πολλαπλά Αρχεία

Δραστηριότητα 1 – Ενιαίο Αρχείο Προγράμματος

Δημιουργήστε ένα αρχείο με όνομα main-to-split.cpp με το παρακάτω περιεχόμενο και εκτελέστε το:

```
#include <iostream>
using namespace std;

void add2Num(int *, int *, int *); // pass by reference
int diff2Num(int, int); // pass by value

int main(){

    int num1=4, num2=1;
    int sum, diff;
    int array[] = {2, 4, 6};
    int sum_array = 0; // store sum of elements in array[]

    for (int i=0; i<3; i++) // loop to find sum of array
        sum_array += array[i];
    cout << "sum_array = " << sum_array << endl;

    // add two numbers
    add2Num(&num1, &num2, &sum); // pass by reference
    cout << "sum of " << num1 << " and " << num2 << " = " << sum << endl;

    // subtract two numbers
    diff = diff2Num(num1, num2); // pass by value
    cout << "difference of " << num1 << " and " << num2 << " = " << diff << endl;
}

// pass by reference
void add2Num (int *a, int *b, int *c){
    *c = *a + *b;
}

// pass by value
int diff2Num(int a, int b){
    return (a-b);
}
```

Δραστηριότητα 2 – Απομονώνοντας το κυρίως πρόγραμμα

Δημιουργήστε ένα αρχείο με όνομα **my_header.hpp** με το παρακάτω περιεχόμενο :

```
#include <iostream>
using namespace std;

void add2Num(int *, int *, int *); // pass by reference
int diff2Num(int, int); // pass by value
```

Δημιουργήστε ένα αρχείο με όνομα **main.cpp** (στον ίδιο φάκελο με το **my_header.cpp**) με το παρακάτω περιεχόμενο :

```
#include "my_header.hpp"

int main(){

    int sum_array = 0; // store sum of elements in array[]

    int num1=4, num2=1;
    int sum, diff;
    int array[] = {2, 4, 6};

    for (int i=0; i<3; i++) // loop to find sum of array
        sum_array += array[i];
    cout << "sum_array = " << sum_array << endl;

    // add two numbers
    add2Num(&num1, &num2, &sum); // pass by reference
    cout << "sum of " << num1 << " and " << num2 << " = " << sum << endl;

    // subtract two numbers
    diff = diff2Num(num1, num2); // pass by value
    cout << "difference of " << num1 << " and " << num2 << " = " << diff << endl;
}
```

Δημιουργήστε ένα αρχείο με όνομα **my_functions.cpp** (στον ίδιο φάκελο με το **main.cpp**) με το παρακάτω περιεχόμενο :

```
#include "my_header.hpp"

void add2Num (int *a, int *b, int *c){
    *c = *a + *b;
}

int diff2Num(int a, int b){
    return (a-b);
}
```

Για να κάνουμε compile πρέπει στο Terminal να εισάγουμε πλέον τη παρακάτω εντολή:

```
g++ main.cpp my_functions.cpp -o pollapla
```

Μετά εκτελούμε το pollapla (./pollapla)

Δραστηριότητα 3 – Προστασία από διπλές εισαγωγές

Αλλάξτε το περιεχόμενο του **my_header.hpp** ως εξής :

```
#pragma once

#include <iostream>
using namespace std;

#define PI 3.14
void add2Num(int *, int *, int *); // pass by reference
int diff2Num(int, int); // pass by value

double circle_perimeter(int);
```

Προσθέστε στο **main.cpp** τον παρακάτω κώδικα :

```
cout<<"Circle Perimeter with radius "<<num1<<" = "<<circle_perimeter(num1)<<endl;
```

Προσθέστε στο **my_functions.cpp** τον παρακάτω κώδικα :

```
double circle_perimeter(int r){
    return 2 * PI * r;
}
```

Κάντε ξανά compile και εκτελέστε να δείτε το αποτέλεσμα.

Το γεγονός ότι προσθήσαμε στο αρχείο επικεφαλίδας **my_header.hpp** τις εντολές **#pragma once** στην αρχή του αρχείου μας προστατεύει από την περίπτωση κυκλικής αναφοράς (το να αναφέρει ένα αρχείο κάτι που έχει ήδη εισαχθεί από προηγούμενο αρχείο).

Δραστηριότητα 4 – Μεταγλώττιση σε βήματα

Μπορούμε να κάνουμε Μεταγλώττιση σε βήματα:

- Δημιουργούμε πρώτα object files (.o) με τις εντολές **g++ -c main.cpp** και **g++ -c my_functions.cpp**
Οι εντολές αυτές θα παράξουν τα αρχεία **main.o** και **my_functions.o** αντίστοιχα
- Στη συνέχεια αξιοποιούμε τα object files για να παράξουμε το εκτελέσιμο πρόγραμμα **myprog** με την εντολή **g++ -o myprog main.o my_functions.o -l.**

Ακολουθώντας τη μεταγλώττιση σε βήματα μπορούμε επίσης να κάνουμε μεταγλώττιση μόνο στα αρχεία που κάνουμε αλλαγές.

Για παράδειγμα αν κάνετε αλλαγές στο **main.cpp** μπορείτε να ενημερώσετε το εκτελέσιμο με τις ακόλουθες εντολές:

```
g++ -c main.cpp
g++ -o myprog main.o my_functions.o -l.
```

Δραστηριότητα 5 – Αυτοματοποίηση μεταγλώττισης με χρήση Makefile

Δημιουργήστε ένα αρχείο με όνομα **Makefile** (στον ίδιο φάκελο με το main.cpp) με το παρακάτω περιεχόμενο :

```
CC=g++
CFLAGS=-I.
DEPS = my_header.hpp
OBJ = main.o my_functions.o

%.o: %.cpp $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)

myprog: $(OBJ)
    $(CC) -o $@ $^ $(CFLAGS)

.PHONY: clean

clean:
    del *.o
```

Για να κάνουμε compile μπορούμε τώρα στο Terminal να εισάγουμε την εντολή **C:\msys64\mingw64\bin\mingw32-make** και θα παραχθεί το εκτελέσιμο **myprog**.

Το αρχείο Makefile δηλώνει τον τρόπο που θα δημιουργηθεί το εκτελέσιμο.

Αρχικά δηλώνονται μια σειρά από σταθερές που χρησιμοποιούνται στη συνέχεια:

- CC για να ορίσουμε τον compiler που θα χρησιμοποιηθεί (g++)
- CFLAGS θέτουμε τιμή -I. ώστε ο compiler να αναζητά τα .hpp αρχεία στον τρέχοντα φάκελο (.)
- DEPS ορίζουμε τη λίστα των αρχείων .hpp από τα οποία εξαρτώνται τα αρχεία .cpp
- OBJs ορίζουμε τη λίστα των object files (.o) που θα χρησιμοποιηθούν στο τελικό βήμα της μεταγλώττισης

Στη συνέχεια ορίζουμε τον τρόπο που παράγονται όλα τα object files (.o)

Η εντολή εκτελείται για κάθε .cpp αρχείο. Έτσι για παράδειγμα για το αρχείο main.cpp εκτελείται η εντολή **g++ -c -o main.o main.cpp -I.**

Τελικά δημιουργείται το εκτελέσιμο **myprog** που αξιοποιεί τα object files (.o) εκτελώντας στην ουσία την εντολή: **g++ -o myprog main.o my_functions.o -I.**

Αν θέλουμε μπορούμε να σβήσουμε όλα τα .o αρχεία με την εντολή

C:\msys64\mingw64\bin\mingw32-make clean

Κάνετε αλλαγές στο main.cpp, προσθέτοντας επιπλέον στοιχεία στον πίνακα array. Στη συνέχεια εκτελέστε την εντολή **C:\msys64\mingw64\bin\mingw32-make** για να ενημερώσετε το εκτελέσιμο myprog.

Μεταβείτε στη σελίδα <https://www.cs.colby.edu/maxwell/courses/tutorials/maketutor/> για περισσότερα Παραδείγματα από Makefiles